



SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

GitHub Profile Analyzer

A Web-Based Platform for Analyzing GitHub Profiles,
Repositories and Developer Activity

Document Version	1.0
Document Status	Final – For Faculty Evaluation
Prepared For	GLA University
Course / Program	B.Tech CSE
Prepared By	Pramit Kumar & 22515000099
Project Guide	Dr. Gautam Mukherjee
Date	18/08/2026



Table of Contents

1. Introduction	4
1.1 Purpose	4
1.2 Document Conventions	4
1.3 Intended Audience and Reading Suggestions	4
1.4 Project Scope	4
1.5 References	4
2. Overall Description	5
2.1 Product Perspective	5
2.2 Product Features (Summary)	5
2.3 User Classes and Characteristics	5
2.4 Operating Environment	5
2.5 Design and Implementation Constraints	6
2.6 Assumptions and Dependencies	6
3. System Features (Functional Requirements)	6
3.1 Module: User Registration & Authentication	6
3.2 Module: Profile Search & Analysis	7
3.3 Module: Repository & Code Insights	7
3.4 Module: Comparison & Leaderboard	8
3.5 Module: Reports, Export & Sharing	8
3.6 Module: Administrator Portal	9
4. External Interface Requirements	9
4.1 User Interfaces	9
4.2 Hardware Interfaces	9
4.3 Software Interfaces	10
4.4 Communication Interfaces	10
5. Use Case Summary	10
6. Non-Functional Requirements	11
7. Software Testing Considerations	12
8. Proposed System Architecture	13
9. Data Model Overview (for Database Planning)	14

10. Future Enhancements (Out of Current Scope)	15
11. Glossary	15

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **GitHub Profile Analyzer**, a web-based application that allows any visitor to enter a GitHub username and instantly receive a rich, visual analysis of that developer's public profile — including repository statistics, programming-language distribution, contribution activity, and a computed "developer score". The document is intended to serve as the single source of truth for the development team, project guide, and evaluation committee regarding what the system will do, who will use it, and the constraints under which it must be built. It is written to be used for requirement analysis, UI/UX design, database and API planning, software testing, and final project documentation.

1.2 Document Conventions

Functional requirements are uniquely identified using the format FR-XX, and are grouped by module. Each requirement is written using the keyword "shall" to indicate a mandatory capability of the system, consistent with IEEE 830 / ISO/IEC/IEEE 29148 conventions. Priority levels used throughout this document are High (must be implemented for a working system), Medium (important but can be simplified if time is short) and Low (optional / future enhancement).

1.3 Intended Audience and Reading Suggestions

- **Students / Development Team** – to understand what to build, module by module.
- **Project Guide / Faculty Evaluator** – to assess completeness, correctness and feasibility of scope.
- **UI/UX Designer** – to design screens based on Section 4 and Section 3 workflows.
- **Database / Backend Developer** – to design schema and APIs based on Section 8 and Section 9.
- **Software Tester** – to derive test cases directly from the functional requirements in Section 3.

1.4 Project Scope

The GitHub Profile Analyzer will allow any visitor to search for a public GitHub username and view an aggregated analysis built from the GitHub REST/GraphQL API: profile summary, repository statistics, star/fork totals, language distribution, and a contribution activity graph. Registered users will be able to save profiles to a personal search history, generate and download PDF reports, compare two profiles side-by-side, and optionally appear on a public developer leaderboard. Administrators will oversee platform usage, manage cached data, and moderate leaderboard content.

The scope is deliberately kept realistic for a college-level team project. The system reads and analyzes *publicly available* GitHub data only — it does not modify any GitHub repository, does not require write access to a user's GitHub account, and is not an official GitHub product. Private repository analysis, organization-level analytics, CI/CD pipeline integration, and AI-generated code-review features are explicitly out of scope for the current version and are listed as future enhancements in Section 10.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Requirements Engineering.
- GitHub REST API v3 and GitHub GraphQL API v4 official documentation.
- React.js, Node.js, Express.js and MongoDB official documentation.
- Project description document / synopsis submitted by the student team.

2. Overall Description

2.1 Product Perspective

GitHub Profile Analyzer is a new, self-contained, independent web application. It is not a modification of GitHub itself and has no affiliation with GitHub, Inc. It follows a standard three-tier web architecture (presentation, application/API, and data layer — detailed in Section 8) that consumes the public GitHub API as an external data source, with a caching layer to respect API rate limits and improve responsiveness.

2.2 Product Features (Summary)

- User registration/login via email or “Sign in with GitHub” OAuth.
- Instant profile analysis by GitHub username: bio, followers, repos, join date.
- Repository insights: stars, forks, open issues, last-updated, language breakdown.
- Contribution activity graph and consistency/streak statistics.
- Automatically computed Developer Score based on activity and repository quality.
- Side-by-side comparison of two GitHub profiles.
- Optional public leaderboard ranking analyzed profiles by Developer Score.
- Downloadable PDF report and embeddable SVG stats badge for READMEs.
- Admin dashboard for usage analytics, cache management and leaderboard moderation.

2.3 User Classes and Characteristics

User Class	Description	Technical Characteristics
Guest Visitor	Anyone browsing the site without an account.	Can search and view a profile analysis; cannot save history or appear on the leaderboard.
Registered User	A developer who creates an account to save work.	Basic-to-moderate technical familiarity; uses saved history, comparisons and PDF export.
Analyzed Profile Owner	The real GitHub user whose public data is analyzed.	May or may not be a registered platform user themselves; opts in/out of leaderboard listing.

User Class	Description	Technical Characteristics
Administrator	Platform owner / maintainer (project team member).	Trusted internal user with full configuration, moderation and analytics access.

2.4 Operating Environment

- **Client side:** Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, tablet or smartphone with an active internet connection.
- **Server side:** Node.js runtime environment hosted on a cloud platform (e.g. Render, Railway, AWS/Azure free tier) or a local server for demonstration.
- **Database:** MongoDB Atlas (cloud) or local MongoDB instance, with an optional Redis instance for API response caching.
- **External Data Source:** GitHub REST API v3 / GraphQL API v4 (public, unauthenticated or OAuth-authenticated requests).

2.5 Design and Implementation Constraints

- Must be developed and demonstrated within a single college semester by a small student team (typically 2–4 members).
- Must primarily use free / open-source technologies to keep the project cost at zero.
- Must respect the GitHub API's rate limits (60 requests/hour unauthenticated, 5,000 requests/hour authenticated) via caching and request batching.
- Must be responsive and usable on both desktop and mobile screen sizes.
- GitHub OAuth login and SVG-badge embedding are treated as important but simplifiable features if time is short.
- The system assumes analysis of individual public GitHub accounts only (organization-level analytics is a future enhancement).

2.6 Assumptions and Dependencies

- Users have basic familiarity with GitHub usernames and web forms.
- The GitHub username entered by the user corresponds to a public profile with public repositories.
- Reliable internet connectivity is available at both client and server ends.
- The GitHub API remains available and its rate-limit policy remains materially unchanged during the project period.
- Email delivery (for OTP/password reset) depends on a third-party SMTP service (e.g. Gmail SMTP / SendGrid free tier).

3. System Features (Functional Requirements)

This section lists the functional requirements of the system, grouped by module. Each requirement can be traced directly to a test case (see Section 7) and a screen (see Section 4).

3.1 Module: User Registration & Authentication

Description: Allows visitors to securely create accounts and log in, with role-based access.

ID	Requirement	Priority
FR-01	The system shall allow a new user to register using name, email, and password.	High
FR-02	The system shall allow a user to sign up / log in using "Sign in with GitHub" OAuth.	High
FR-03	The system shall validate email format and enforce a minimum password strength on registration.	High
FR-04	The system shall authenticate registered users via email/password or GitHub OAuth login.	High
FR-05	The system shall issue a secure session/token (e.g. JWT) upon successful login.	High
FR-06	The system shall allow a user to reset a forgotten password via a one-time email link/OTP.	Medium
FR-07	The system shall restrict access to module features based on the logged-in user's role (User/Admin).	High
FR-08	The system shall allow a logged-in user to log out, invalidating the active session.	Medium

3.2 Module: Profile Search & Analysis

Description: The core analysis experience — the heart of the product.

ID	Requirement	Priority
FR-09	The system shall allow any visitor to search for a GitHub profile by entering a valid GitHub username.	High
FR-10	The system shall fetch and display the profile summary (avatar, bio, location, followers, following, public repo count, join date) from the GitHub API.	High
FR-11	The system shall calculate and display aggregate metrics: total stars received, total forks, and total public repositories.	High
FR-12	The system shall display a language-distribution chart summarizing the programming languages used across the user's repositories.	High
FR-13	The system shall display a contribution activity graph (commits/contributions per day or week over the past year).	Medium
FR-14	The system shall compute and display the user's longest and current contribution streak.	Medium

ID	Requirement	Priority
FR-15	The system shall allow a logged-in user to save an analyzed profile to their personal search history.	Medium
FR-16	The system shall display the user's top repositories ranked by stars and forks.	High
FR-17	The system shall gracefully handle and display a clear message when a searched username does not exist on GitHub.	High

3.3 Module: Repository & Code Insights

Description: Deeper, repository-level analytics for a searched profile.

ID	Requirement	Priority
FR-18	The system shall list all public repositories of the analyzed profile with stars, forks, open issues and last-updated date.	High
FR-19	The system shall allow the viewer to filter/sort the repository list by stars, language, or last-updated date.	Medium
FR-20	The system shall display repository-level insights such as commit frequency and primary contributors, where available via the API.	Low
FR-21	The system shall detect and visually flag inactive or archived repositories (e.g. no commits in 12+ months).	Low
FR-22	The system shall compute a weighted "Developer Score" based on activity consistency, star/fork totals, and repository quality signals.	Medium

3.4 Module: Comparison & Leaderboard

ID	Requirement	Priority
FR-23	The system shall allow a user to compare two GitHub profiles side-by-side.	Medium
FR-24	The system shall display comparative charts for stars, followers, repositories and top languages between the two compared profiles.	Medium
FR-25	The system shall maintain an optional public leaderboard ranking analyzed profiles by Developer Score.	Low
FR-26	The system shall allow a registered profile owner to opt in or opt out of leaderboard visibility.	Medium

3.5 Module: Reports, Export & Sharing

ID	Requirement	Priority
FR-27	The system shall allow a user to generate and download a PDF summary report of an analyzed profile.	Medium
FR-28	The system shall generate a shareable public link/card summarizing a profile's analysis.	Low
FR-29	The system shall generate an embeddable SVG stats badge/widget that a user can place in their own README file.	Low

3.6 Module: Administrator Portal

ID	Requirement	Priority
FR-30	The system shall allow the Admin to view platform usage statistics (searches performed, active users, top-searched profiles).	Medium
FR-31	The system shall allow the Admin to view and clear cached GitHub API data.	Medium
FR-32	The system shall allow the Admin to remove or hide a profile from the public leaderboard (moderation).	Low
FR-33	The system shall allow the Admin to configure GitHub API rate-limit handling settings (e.g. cache duration).	Low

4. External Interface Requirements

4.1 User Interfaces

- **Landing / Home Page** – product introduction, username search bar, sample analysis preview, login/register call-to-action.

- **Profile Analysis Dashboard** – profile summary card, key metric tiles, language-distribution chart, contribution graph, top repositories.
- **Repository Insights Page** – full sortable/filterable repository list with per-repo details.
- **Comparison Page** – two-profile input and side-by-side comparative charts.
- **Leaderboard Page** – ranked list of opted-in profiles by Developer Score.
- **User Dashboard** – saved search history, generated reports, account/profile settings.
- **Admin Dashboard** – usage analytics, cache management, leaderboard moderation.
- All screens shall follow a consistent, clean, responsive layout (see Section 6.3) using a modern, developer-tool-appropriate colour palette (dark/light theme support recommended).

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard client devices (desktop/laptop/tablet/smartphone) with a working internet connection, and on any commodity server/cloud instance capable of running Node.js and MongoDB.

4.3 Software Interfaces

Component	Technology
Frontend	React.js (with React Router, Axios, and a component/chart library such as Tailwind CSS + Recharts/Chart.js)
Backend / API	Node.js with Express.js – RESTful JSON APIs
Database	MongoDB (Mongoose ODM) – NoSQL document store
Caching Layer	Redis (or in-memory/MongoDB TTL cache) for GitHub API response caching
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing; OAuth 2.0 for “Sign in with GitHub”
External API	GitHub REST API v3 and/or GraphQL API v4 for profile, repository and contribution data
Email Service	SMTP service (e.g. Nodemailer with Gmail/SendGrid) for notifications and password reset
Hosting (suggested)	Frontend – Vercel/Netlify; Backend – Render/Railway; Database – MongoDB Atlas

4.4 Communication Interfaces

- All client–server communication shall occur over HTTPS using RESTful JSON APIs.
- All communication with the GitHub API shall occur over HTTPS, using an authenticated App/Personal Access Token where possible to obtain the higher rate-limit tier.
- Outbound email notifications shall be sent via the SMTP protocol through the configured email service.

- (Future phase) Webhook subscriptions may be integrated to receive near-real-time repository update events.

5. Use Case Summary

The table below summarizes the primary use cases per actor. Detailed step-by-step use-case descriptions (pre-conditions, main flow, alternate flow) can be expanded from this list during design if required by the evaluation format.

Actor	Use Case	Brief Description
Guest / User	Search GitHub Profile	Enter a GitHub username to generate an analysis.
Guest / User	Register / Login	Create an account or sign in (email or GitHub OAuth).
User	View Profile Analysis	See stats, languages, contribution graph and Developer Score.
User	Explore Repositories	Browse, filter and sort a profile's public repositories.
User	Compare Profiles	Compare two GitHub usernames side-by-side.
User	Save / View History	Save analyzed profiles and revisit past searches.
User	Export Report	Download a PDF summary of an analyzed profile.
User	Manage Leaderboard Visibility	Opt in/out of the public developer leaderboard.
Admin	View Usage Analytics	Monitor searches performed and active users.
Admin	Manage Cache	View and clear cached GitHub API data.
Admin	Moderate Leaderboard	Remove or hide flagged profiles from the leaderboard.

5.1 Primary Flow – Profile Analysis

1) Visitor lands on the home page → 2) Enters a GitHub username in the search bar → 3) System validates the username exists via the GitHub API → 4) System fetches profile, repository and contribution data (using cached data if recently fetched) → 5) System computes aggregate metrics and the Developer Score → 6) Dashboard renders the profile summary, charts and top repositories → 7) If logged in, the visitor may save the result to history or export it as a PDF report.

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load any core page within 3 seconds under normal load on a standard broadband connection.	High
NFR-02	The system shall support at least 50–100 concurrent users without noticeable performance degradation (adequate for a college demo/pilot).	Medium
NFR-03	A previously analyzed profile shall be served from cache within 1 second when re-requested within the cache validity window.	Medium
NFR-04	The system shall queue or throttle outgoing GitHub API requests to stay within the applicable rate limit.	High

6.2 Security Requirements

ID	Requirement	Priority
NFR-05	Passwords shall be stored using a one-way hashing algorithm (e.g. bcrypt); plain-text passwords shall never be stored.	High
NFR-06	GitHub OAuth tokens shall be stored encrypted at rest and never exposed to the client.	High
NFR-07	All API endpoints that access personal or saved-history data shall require a valid authentication token.	High
NFR-08	Role-based authorization shall ensure a standard User cannot access Admin-only endpoints.	High
NFR-09	All data in transit shall be encrypted using HTTPS/TLS.	High
NFR-10	User input (usernames, search queries) shall be validated and sanitized on both client and server to prevent injection attacks.	High

6.3 Usability

- The interface shall be simple enough for a first-time visitor to analyze a profile in under 3 clicks/taps.
- The system shall be fully responsive across desktop, tablet and mobile screen sizes.
- Error and success messages (e.g. “user not found”, “rate limit reached”) shall be clear, specific, and shown close to the relevant form field/action.

6.4 Reliability & Availability

- The system shall target 99% uptime during the demonstration/evaluation period.

- Saved user data (history, comparisons, account details) shall not be lost due to a single server restart (persisted in the database, not in memory).
- The system shall continue to serve previously cached analyses if the GitHub API is temporarily unreachable, with a clear “stale data” indicator.

6.5 Maintainability & Portability

- The codebase shall follow a modular MVC-style structure (routes/controllers/models on the backend, reusable components on the frontend) to ease future maintenance and extension by new student contributors.
- The system shall run on any OS capable of hosting Node.js and MongoDB (Windows, Linux, macOS), and shall be accessible from any modern browser without requiring a native app install.

6.6 Scalability

Although scoped for a moderate demo-level user base in the academic version, the data model and API design shall avoid hard-coded assumptions about a single caching node or a single GitHub API token, so that a production deployment (e.g. multiple API tokens, distributed cache) can be added without a full redesign.

7. Software Testing Considerations

Each functional requirement in Section 3 shall map to at least one test case. The table below gives representative examples to guide the team's test plan; the full test case document can be maintained separately (e.g. in a spreadsheet) using this format.

Test ID	Req.	Test Scenario	Expected Result
TC-01	FR-09	Search a valid, existing GitHub username	Profile summary and metrics load and render correctly.
TC-02	FR-17	Search a non-existent GitHub username	System rejects with a clear "user not found" message.
TC-03	FR-11	Verify total stars/forks calculation	Displayed totals match the sum across all fetched public repositories.
TC-04	FR-07	Standard user attempts to access Admin-only page	Access denied (403) / redirected to login.
TC-05	FR-03	Register with a weak password	Registration blocked with validation message.
TC-06	NFR-04	Trigger repeated searches to approach the API rate limit	System serves cached data or shows a graceful rate-limit notice instead of failing.

Recommended testing types: Unit testing (backend controllers/validators using a framework such as Jest/Mocha), Integration testing (API endpoints with Postman or Jest+Supertest, including mocked GitHub API responses), and Manual UI/UX testing across at least two browsers and one mobile viewport before final submission.

8. Proposed System Architecture

Given the team's familiarity and the project's realistic academic scope, a MERN stack (MongoDB, Express.js, React.js, Node.js) three-tier architecture is recommended, with a lightweight caching layer sitting between the application layer and the external GitHub API. This keeps the project fully JavaScript-based end-to-end, is well documented, free to host on student-friendly free tiers, and maps cleanly onto the module breakdown in Section 3.

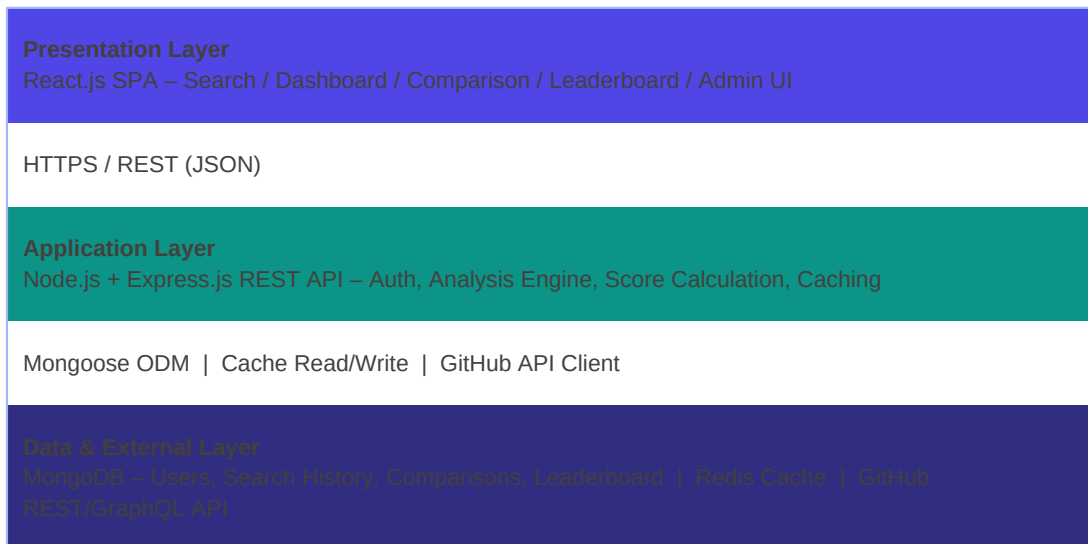


Figure 8.1: Three-tier architecture of the GitHub Profile Analyzer.

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, React Router, Axios, Tailwind CSS, Recharts / Chart.js
Backend	Node.js, Express.js, JWT, bcrypt, Passport.js (GitHub OAuth), Nodemailer
Database	MongoDB with Mongoose
Caching	Redis (or MongoDB TTL collection as a lightweight alternative)
External API	GitHub REST API v3 / GraphQL API v4
Version Control	Git & GitHub
Testing	Postman (API testing), Jest (unit testing)
Deployment (suggested)	Vercel/Netlify (frontend), Render/Railway (backend), MongoDB Atlas (database)

9. Data Model Overview (for Database Planning)

The following core entities and relationships are proposed as a starting point for schema design. This is intentionally kept minimal and realistic for a student project; fields can be extended during implementation.

9.1 Core Entities

Entity	Key Attributes
User	_id, name, email, passwordHash, githubId (nullable), role (user/admin), createdAt
SearchHistory	_id, userId, searchedUsername, snapshotData (cached JSON), analyzedAt
GitHubProfileCache	_id, username, profileData, repoData, contributionData, fetchedAt, expiresAt
Comparison	_id, userId, usernameA, usernameB, resultSnapshot, createdAt
LeaderboardEntry	_id, username, developerScore, isVisible, updatedAt
Report	_id, userId, username, fileUrl, generatedAt

9.2 Key Relationships

- A **User** has many **SearchHistory** entries and many **Comparison** entries (one-to-many).
- A **SearchHistory** entry references one cached **GitHubProfileCache** snapshot (many-to-one).
- A **GitHubProfileCache** record may have at most one corresponding **LeaderboardEntry** (one-to-one, created when a profile is analyzed and opted in).
- A **User** can generate many **Report** records, each tied to one analyzed username (one-to-many).
- A **Comparison** references two GitHub usernames, each resolvable to a **GitHubProfileCache** record.

10. Future Enhancements (Out of Current Scope)

- AI-generated profile summary / resume-style highlights.
- Organization-level and team analytics.
- GitHub webhook integration for near-real-time updates.
- Historical trend tracking (score/stars over time, with charts).
- Browser extension to overlay analyzer stats on GitHub profile pages directly.
- Public API for third-party developers to query analysis data.
- Gamified achievement badges beyond the basic leaderboard.

11. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer – a web API architecture style
GraphQL	A query language for APIs; GitHub exposes a GraphQL API v4 endpoint
JWT	JSON Web Token – used for stateless authentication
OAuth	An open standard for delegated authorization (used for “Sign in with GitHub”)
MERN	MongoDB, Express.js, React.js, Node.js – a full JavaScript web stack
CRUD	Create, Read, Update, Delete – basic data operations
OTP	One-Time Password
Rate Limit	The maximum number of API requests GitHub allows in a given time window
Developer Score	A computed metric summarizing a profile's activity, consistency and repository quality
UI/UX	User Interface / User Experience

End of Document – GitHub Profile Analyzer SRS v1.0